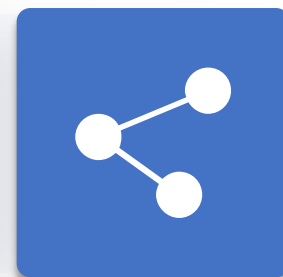


人工智能引论

第3章 状态空间与无信息搜索¹

授课教师：李静瑶²





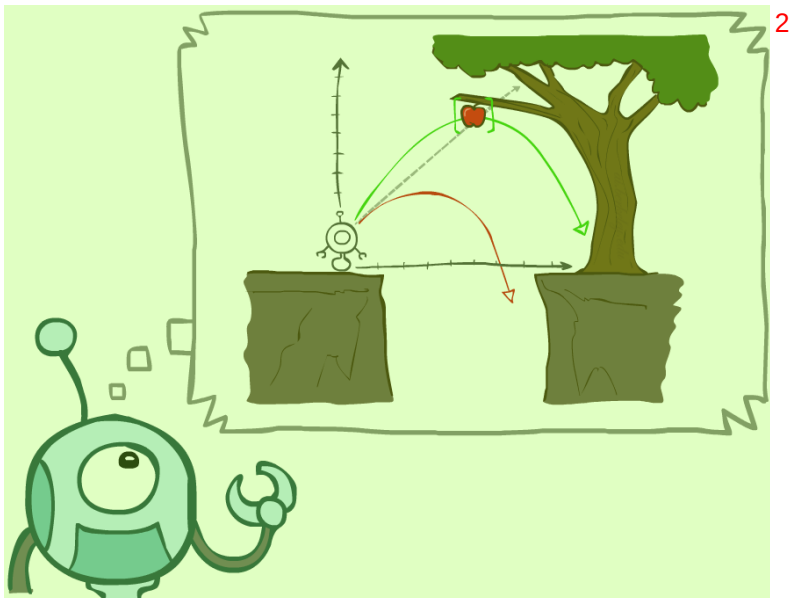
录¹

1 问题求解智能体²

2 问题实例³

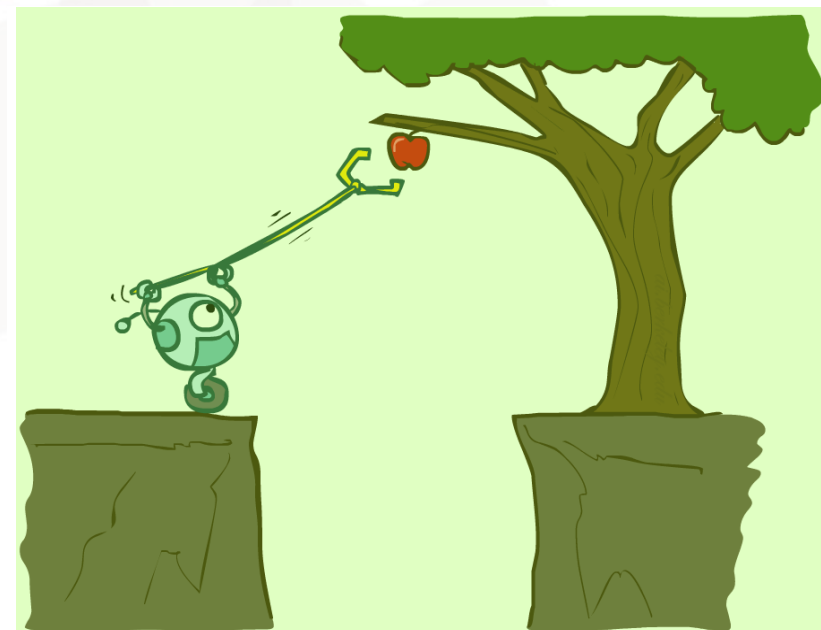
3 搜索算法⁴

4 无信息搜索策略⁵



人工智能的目标：专注于研究和构建具有合理的行为的智能体³

问题求解智能体：考虑一个形成通往目标状态路径的动作序列，它所进行的计算过程被称为搜索⁴



问题求解智能体¹



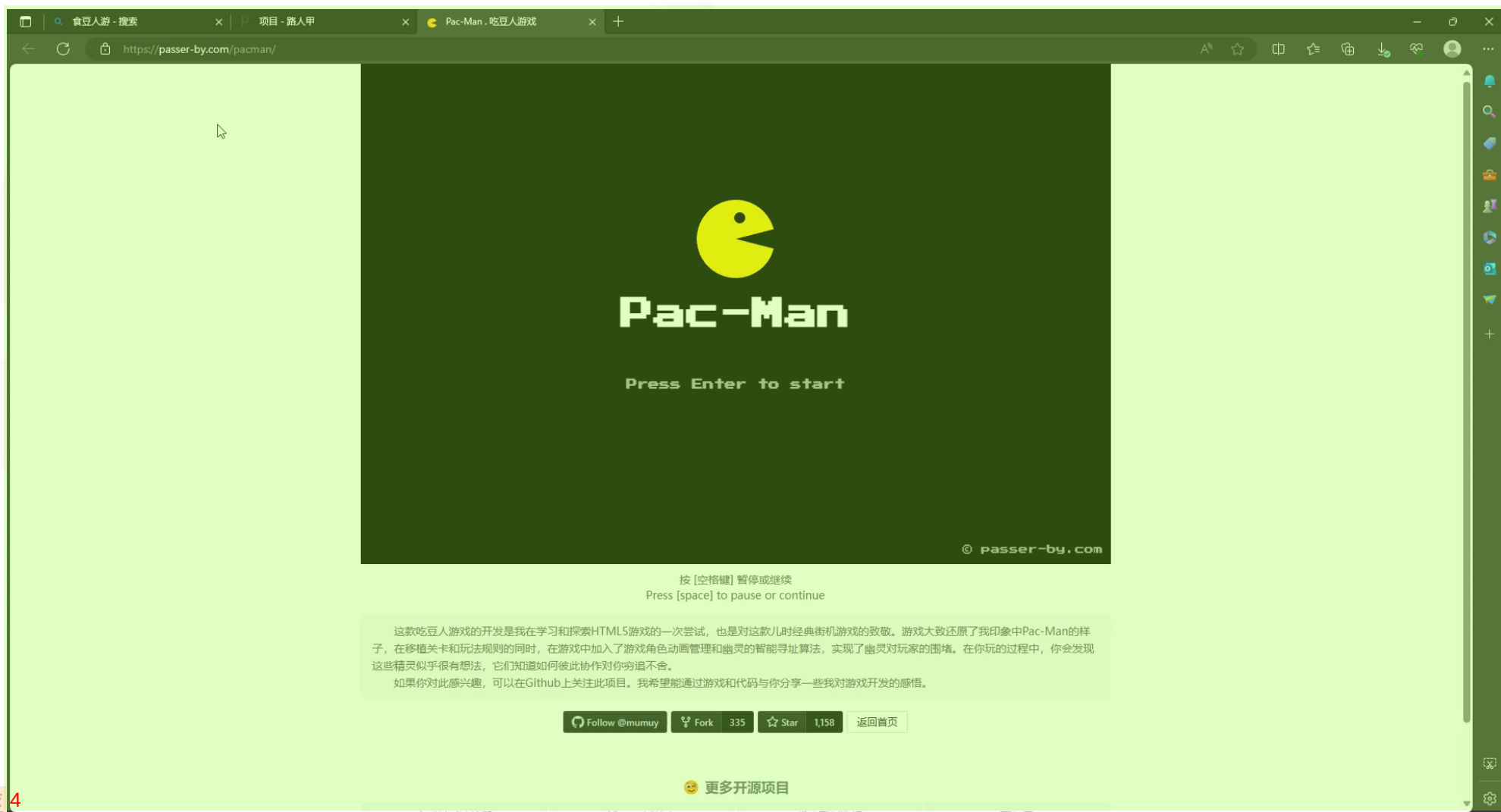
问题求解智能体²

- 规划型智能体，一种基于目标的智能体³
- 问题形式化+目标形式化（可以形式化表示目标并判断目标是否实现）⁴
- 模型知道环境对动作会做出什么反应，考虑一系列动作的结果（搜索）⁵
- 执行（解中的动作）⁶

问题求解智能体¹



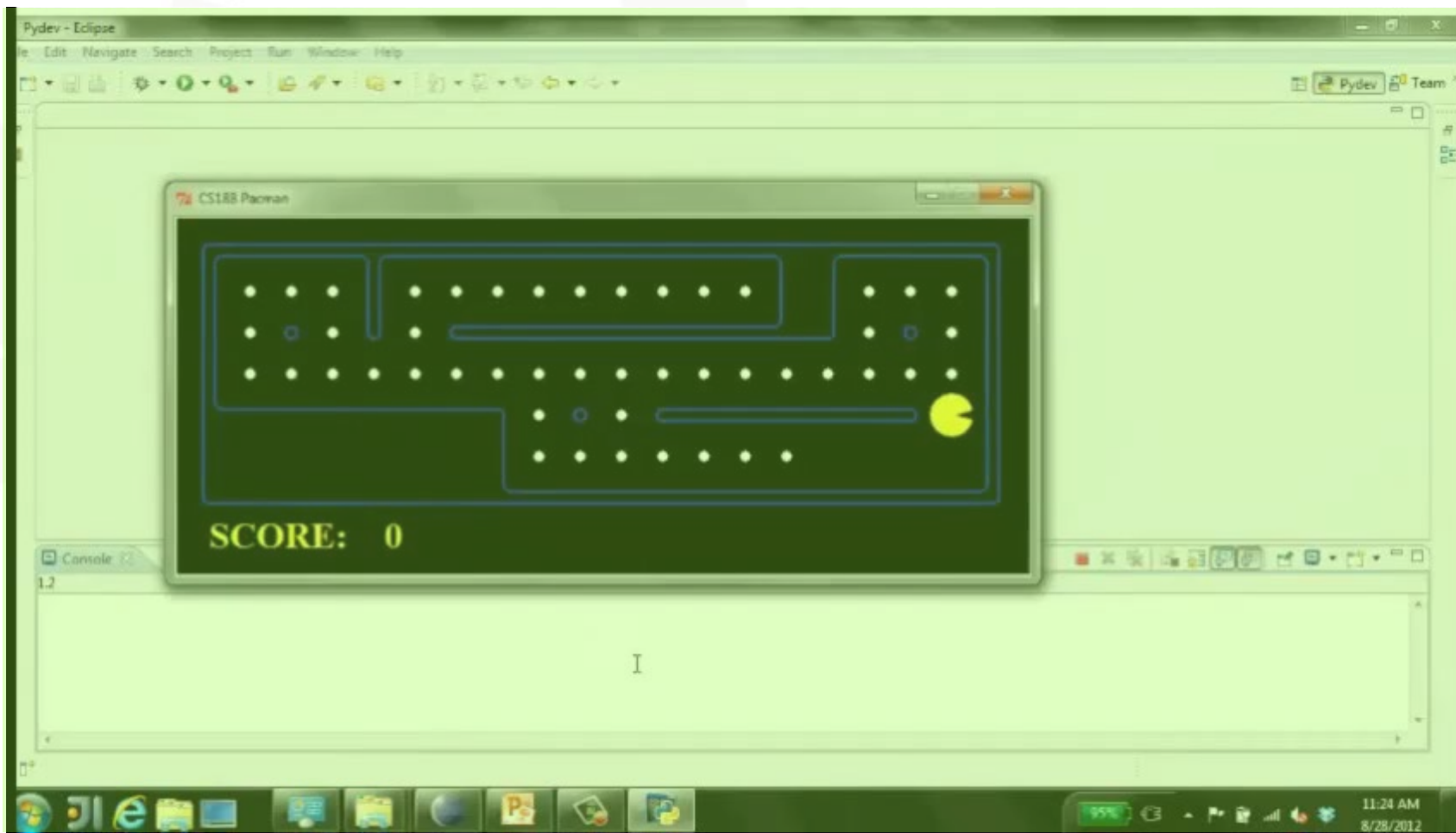
食豆人游戏²



问题求解智能体¹



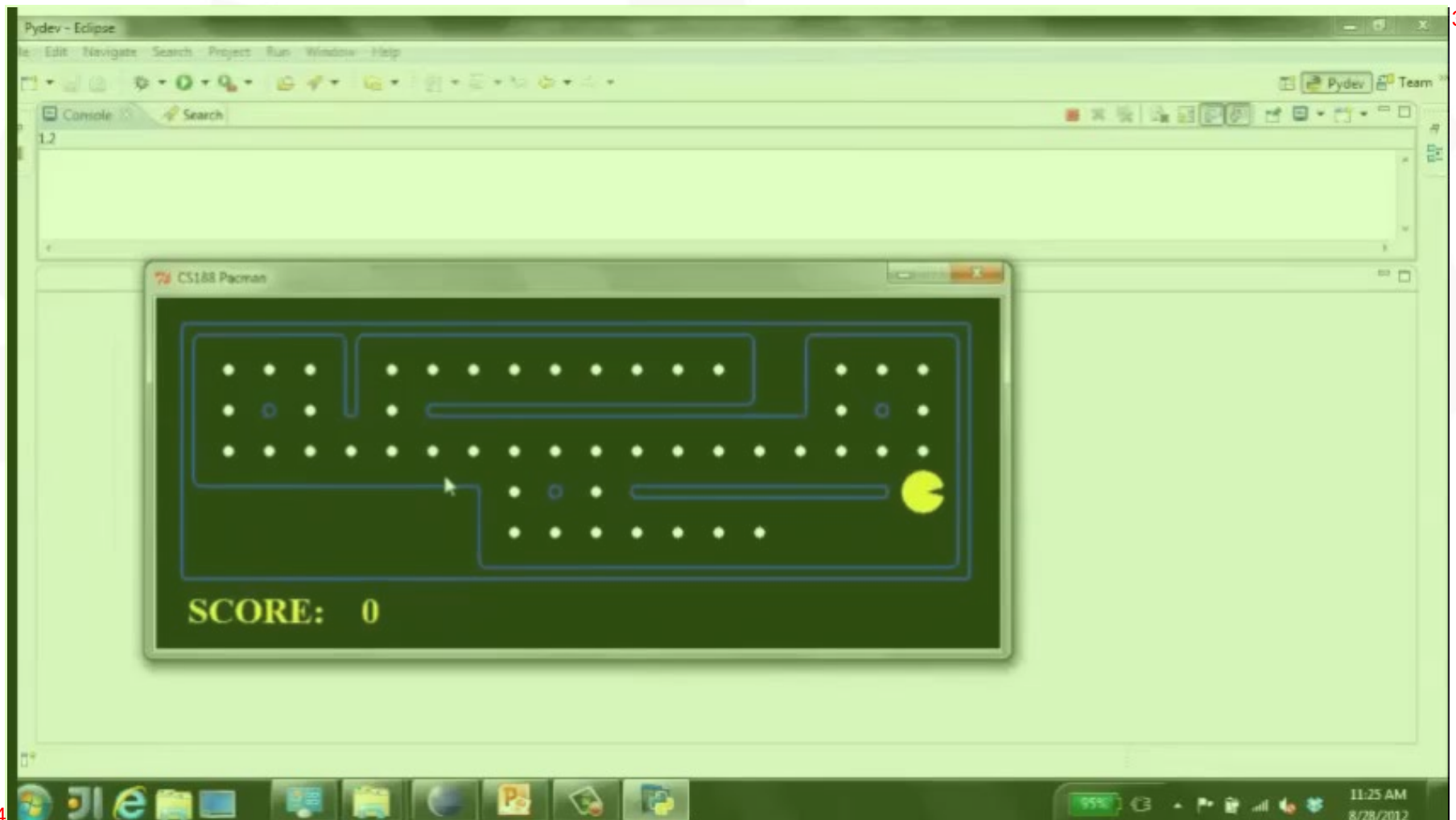
食豆人游戏：吃掉最近的豆子（计算每一步最优路径）²



问题求解智能体¹



食豆人游戏：计算整体最优路径，然后执行²





问题求解智能体¹

搜索问题定义²

- 智能体的初始状态³

// 初始状态⁸

- 目标测试函数⁴

// 目标状态⁹

- 智能体的可执行动作⁵

// 候选动作集¹⁰

- 转移模型⁶

// 状态转移函数¹¹

- 路径代价函数⁷

// 评价函数¹²

问题的解：从初始状态到目标状态的动作序列¹³

- 解的质量由路径代价函数度量¹⁴

- 具有最小路径代价值的解即为最优解¹⁵



例1. 唐纳德·克努斯猜想 (Donald Knuth)²

例2. 真空吸尘器世界³

例3. 八数码问题⁴

例4. 八皇后问题⁵

例5. 路径规划问题⁶

例6. 机器人装配问题⁷

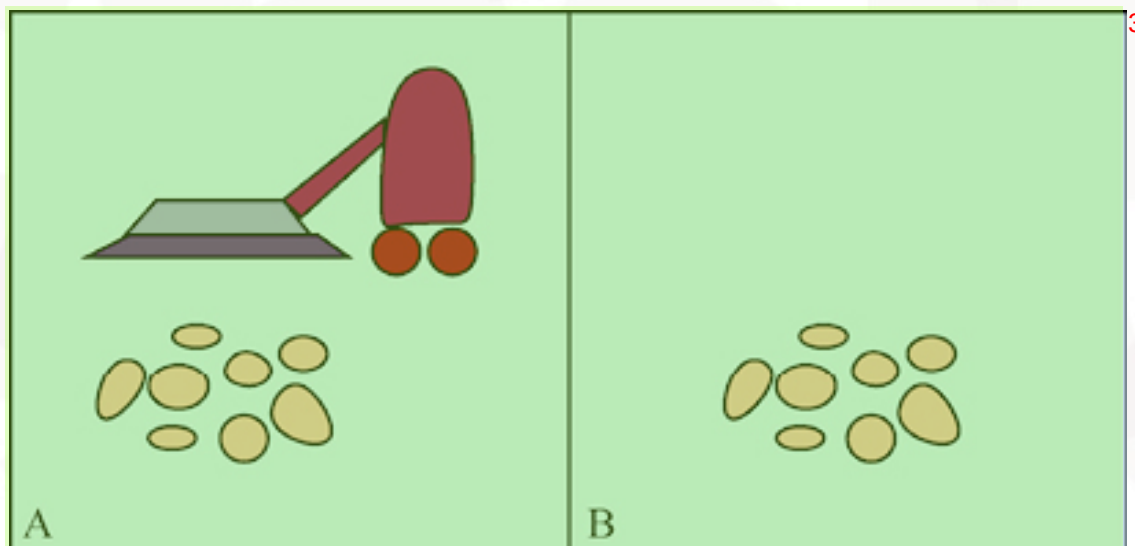


例1. 唐纳德·克努斯猜想，只用数字4，一个由阶乘、开方、取整构造的操作序列可以得到任意正整数。²

- 状态：正整数³
- 初始状态：4⁴
- 目标状态：指定的正整数⁵
- 动作：阶乘、开方、取整⁶
- 路径代价函数：操作数最少...⁷

A green rectangular box containing the mathematical expression $\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} = 5$. The expression shows five nested square roots applied to the factorial of 4 factorial, resulting in the integer 5.

正整数5可以通过如上操作序列得到⁹

例2. 真空吸尘器世界²

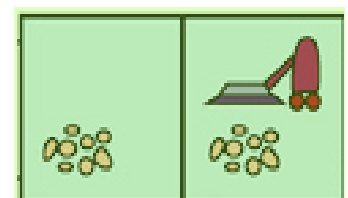
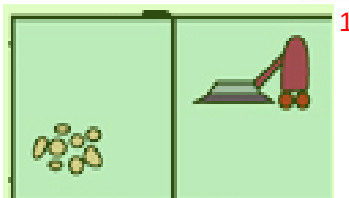
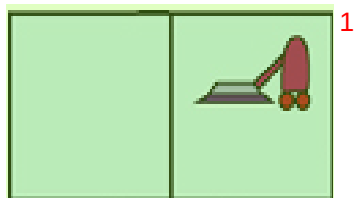
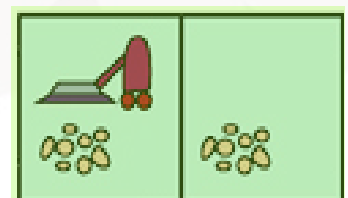
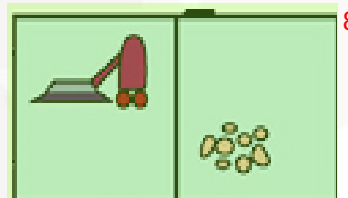
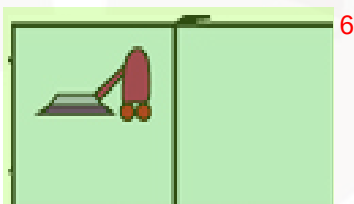
例2. 真空吸尘器世界²

- 状态空间：由Agent位置和房间干净与否确定³

▸ 若Agent的位置有2个，每个房间可能干净或不干净，则可能状态数为⁴

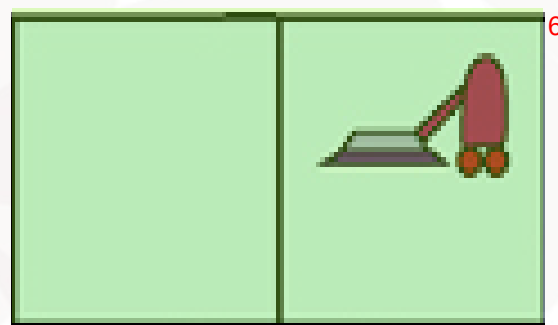
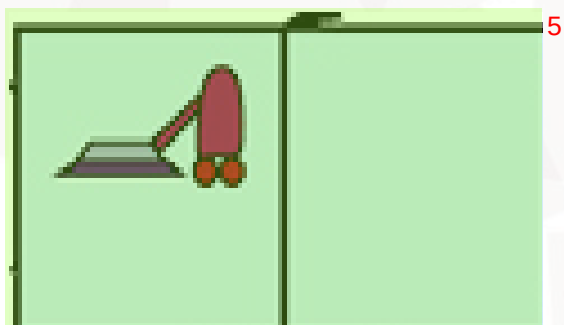
$$2 \times 2^2 = 8$$

▸ 对于具有 n 个房间的大型环境，可能状态数为 $n \times 2^n$ ⁵



例2. 真空吸尘器世界²

- 初始状态：任何状态均可³
- 目标状态：所有房间都干净⁴



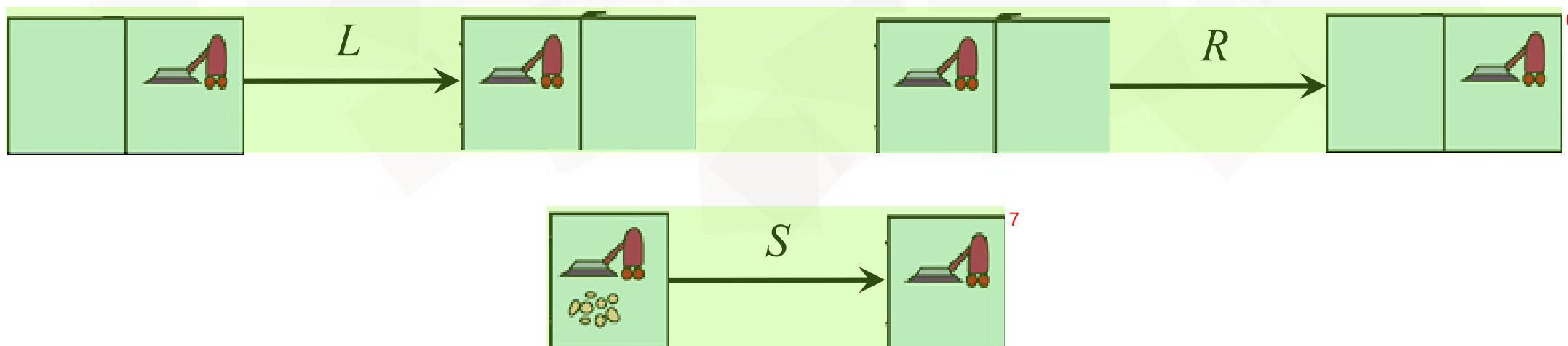
问题实例¹

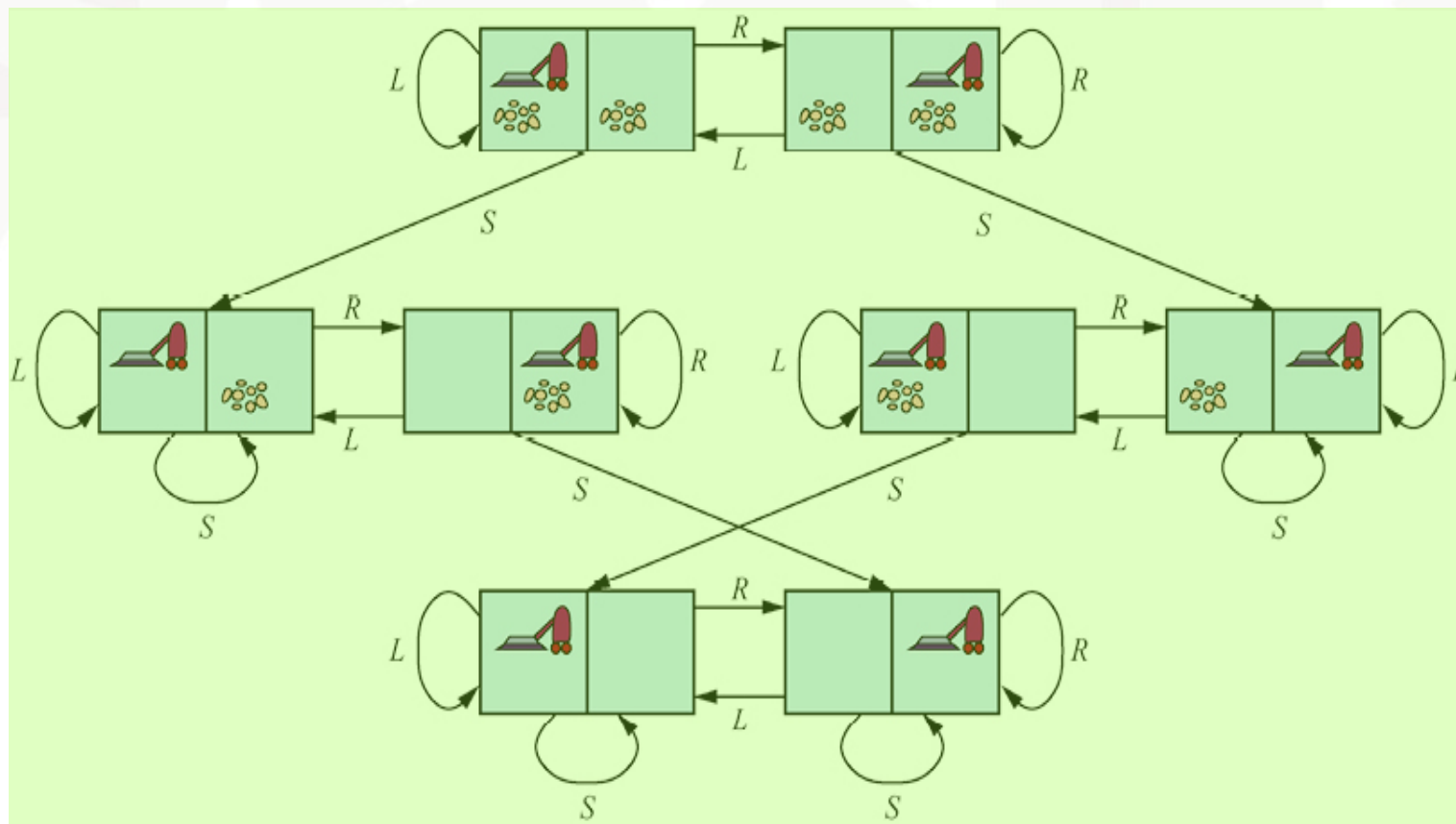
例2. 真空吸尘器世界²

- 可执行动作：吸尘器Agent的动作³

- Left、Right、Suck⁴

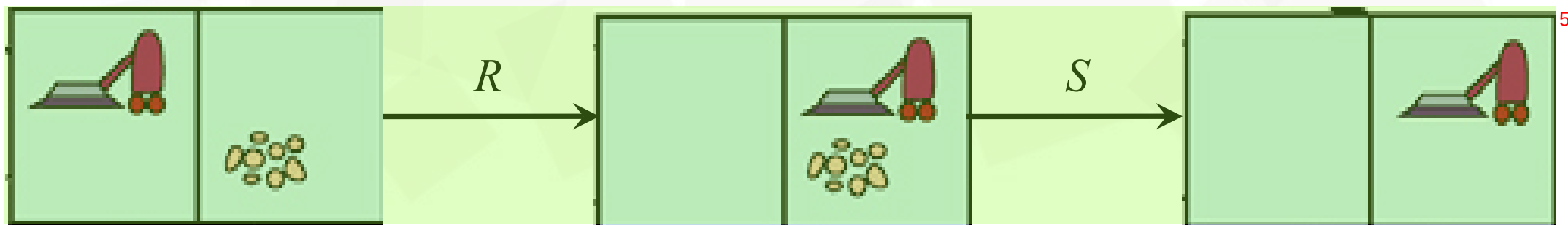
- 大型环境中Up、Down⁵



例2. 真空吸尘器世界²• 状态转移函数³

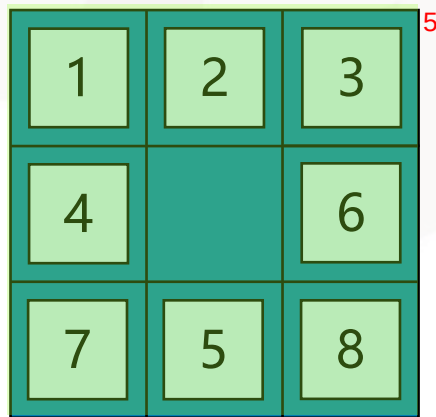
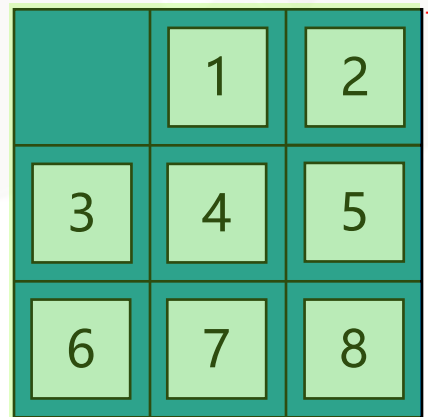
例2. 真空吸尘器世界²• 路径代价函数³

- ▶ 若每个动作的代价值为 1，则路径的代价值为路径中包含的动作数⁴



例3. 八数码问题²

- 一个 3×3 的棋盘上有8个数字棋子和一个空格。与空格相邻的棋子可以滑动到空格中³
- 游戏目标是要达到一个特定的状态⁴

开始状态⁶目标状态⁸

问题实例¹

例3. 八数码问题²

- 状态：描述8个棋子在棋盘上的分布³
 - 初始状态：任何状态均可⁴
 - 目标条件：检测是否为给定的目标状态⁵

1	2	3
4		6
7	5	8

开始状态⁷

1	2	3
4	6	
7	5	8

⁸

■ ■ ■

1	2	3
4	6	8
7	5	

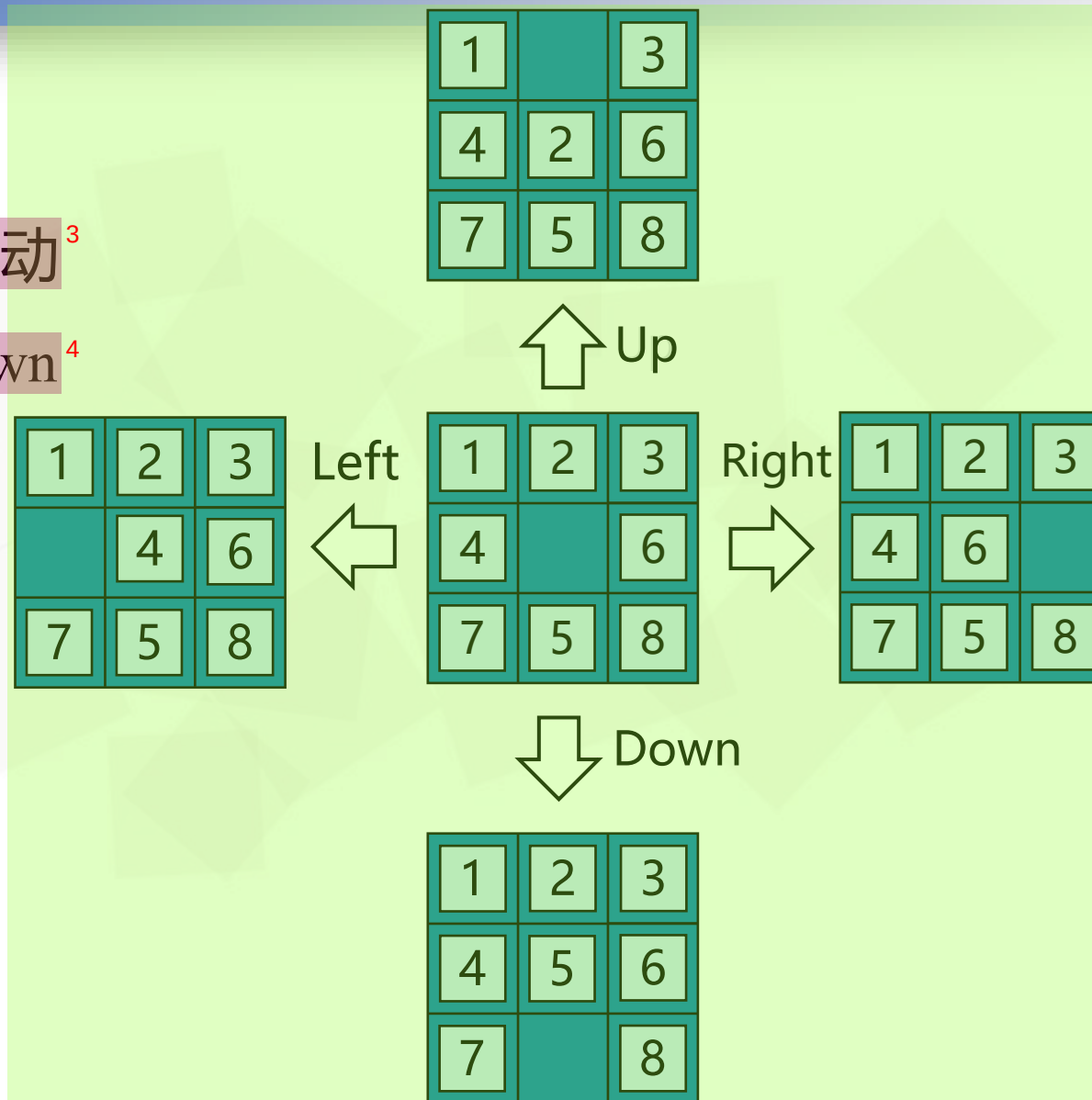
⁹

目标状态¹⁰

例3. 八数码问题²

- 动作：每个棋子或空格的滑动³

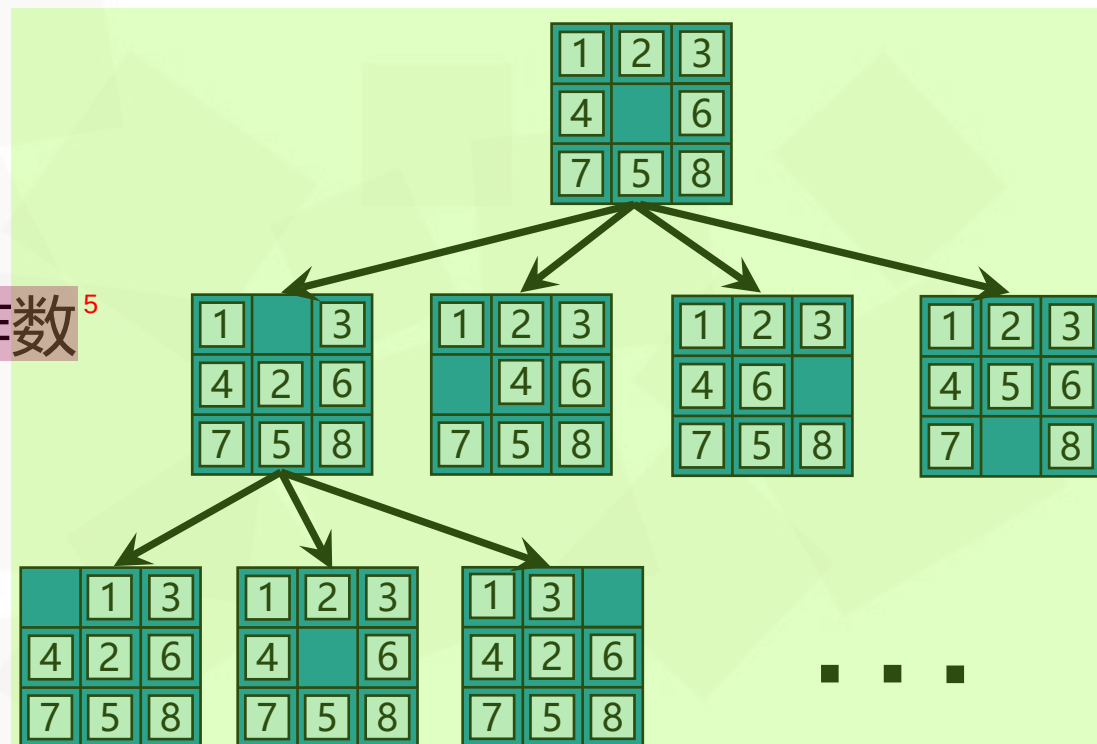
Left、Right、Up、Down⁴



例3. 八数码问题²

- 状态转移函数 (后继函数)³
- 路径代价函数⁴

▸ 路径代价值为路径中的动作数⁵

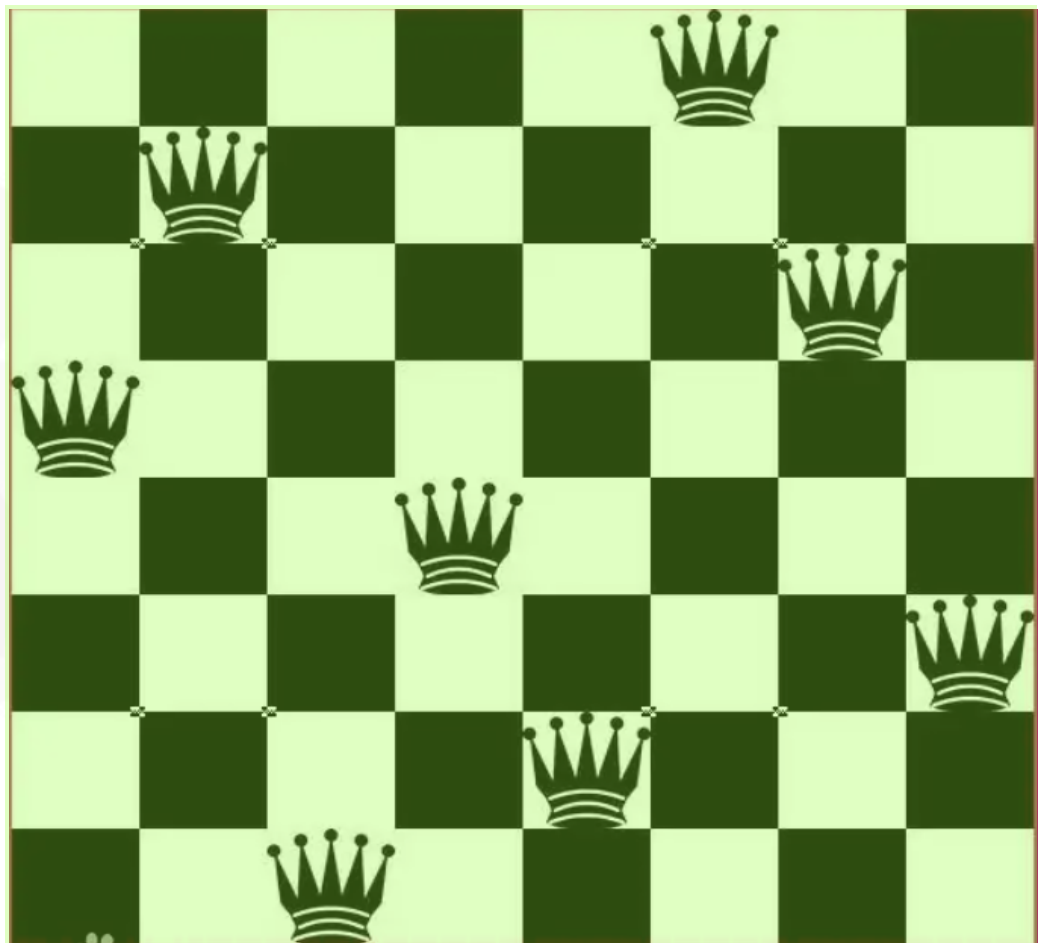


例3. 八数码问题²

- 八数码问题有 $9!/2=181440$ 个可达状态³
- 15数码问题有约1.3万亿个状态，利用最好的搜索算法求解一个随机实例的最优解需要几毫秒⁴
- 24数码问题有约 10^{25} 个状态，利用最好的搜索算法求解一个随机实例的最优解需要几个小时⁵

例4. 八皇后问题²

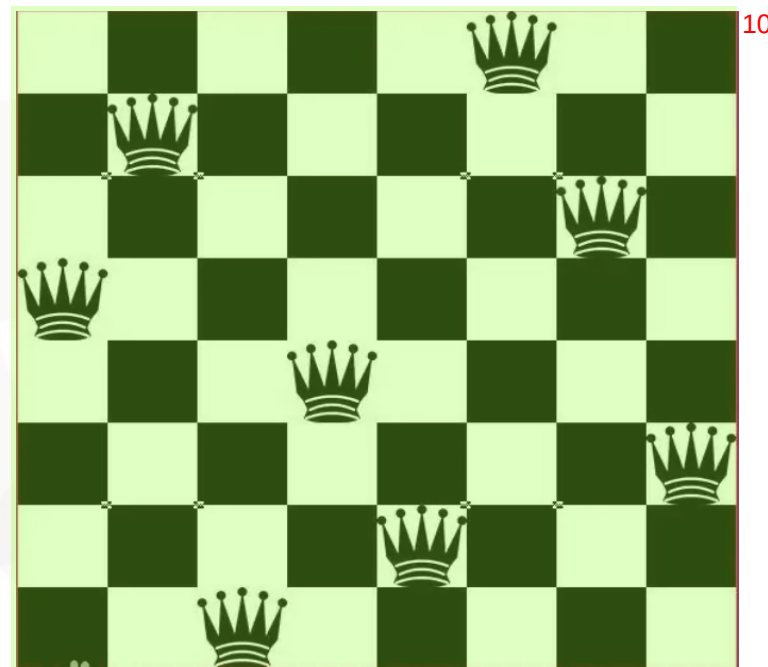
- 目标是在国际象棋棋盘上放置8个皇后,³使得任何一个皇后都不会攻击到其他任一皇后
- 注: 皇后可以攻击和它在同一行、同⁴一列或者同一对角线的任何棋子



问题实例¹

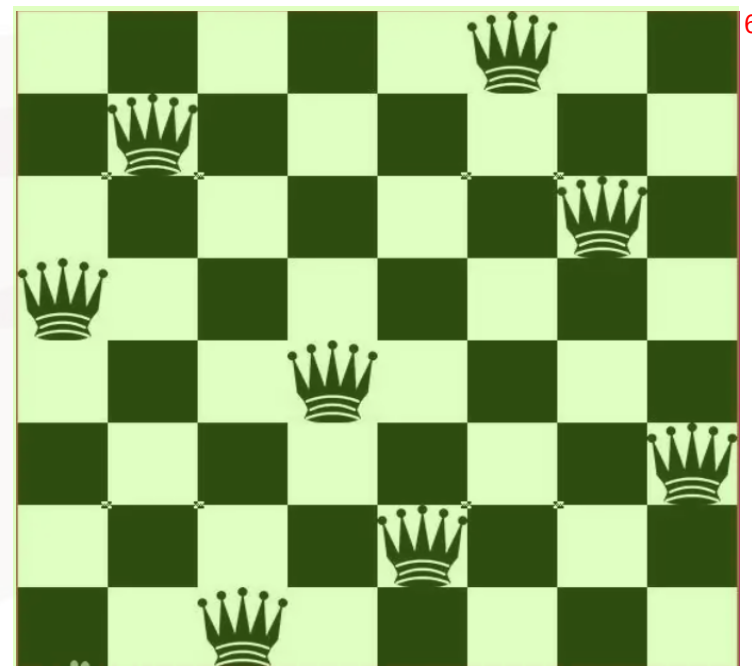
例4. 八皇后问题²

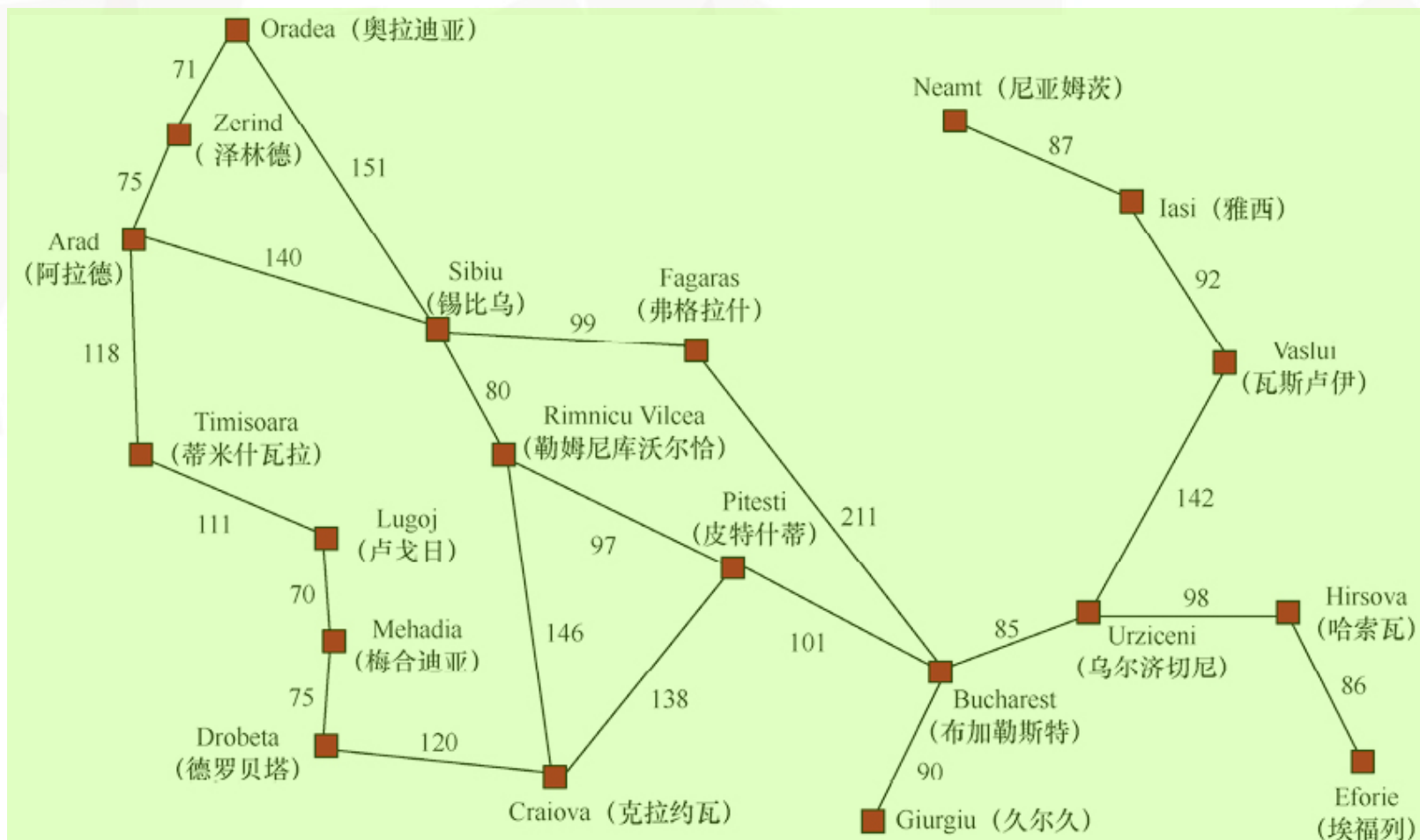
- 状态：描述0-8个皇后在棋盘上的分布³
 - 初始状态：棋盘上没有皇后⁴
 - 目标条件：棋盘上放置8个皇后，无法互相攻击⁵
- 动作：在任一空位放置一个皇后⁶
- 状态转移函数⁷
- 路径代价函数：无需考虑（合法即可）⁸
- 上述描述需要考虑 $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$ 个可能序列⁹



例4. 八皇后问题的改进描述²

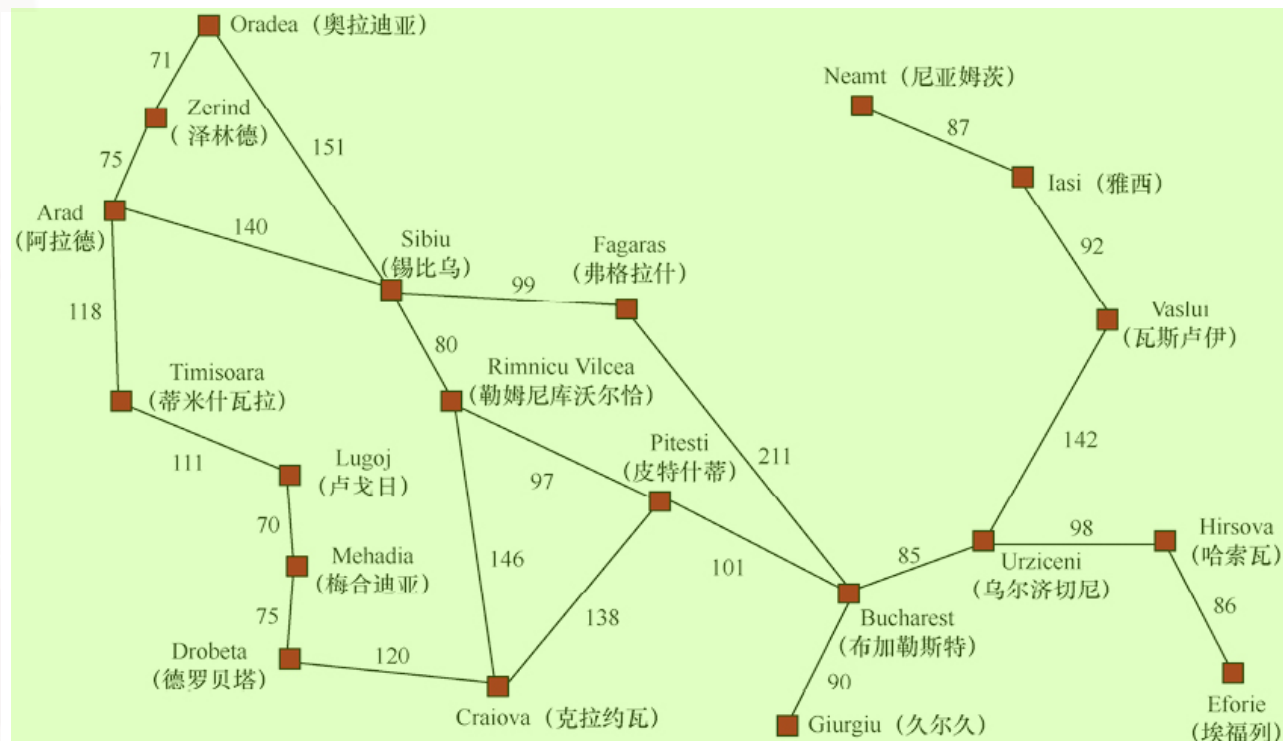
- 状态：描述0-8个皇后在棋盘上的分布，满足最左侧每列放置一个皇后，无法相互攻击³
- 动作：在最左侧空列中的任一空位放置一个皇后，无法相互攻击⁴
- 对于100皇后问题，状态空间从约 10^{400} 个状态减少到约 10^{52} 个状态⁵

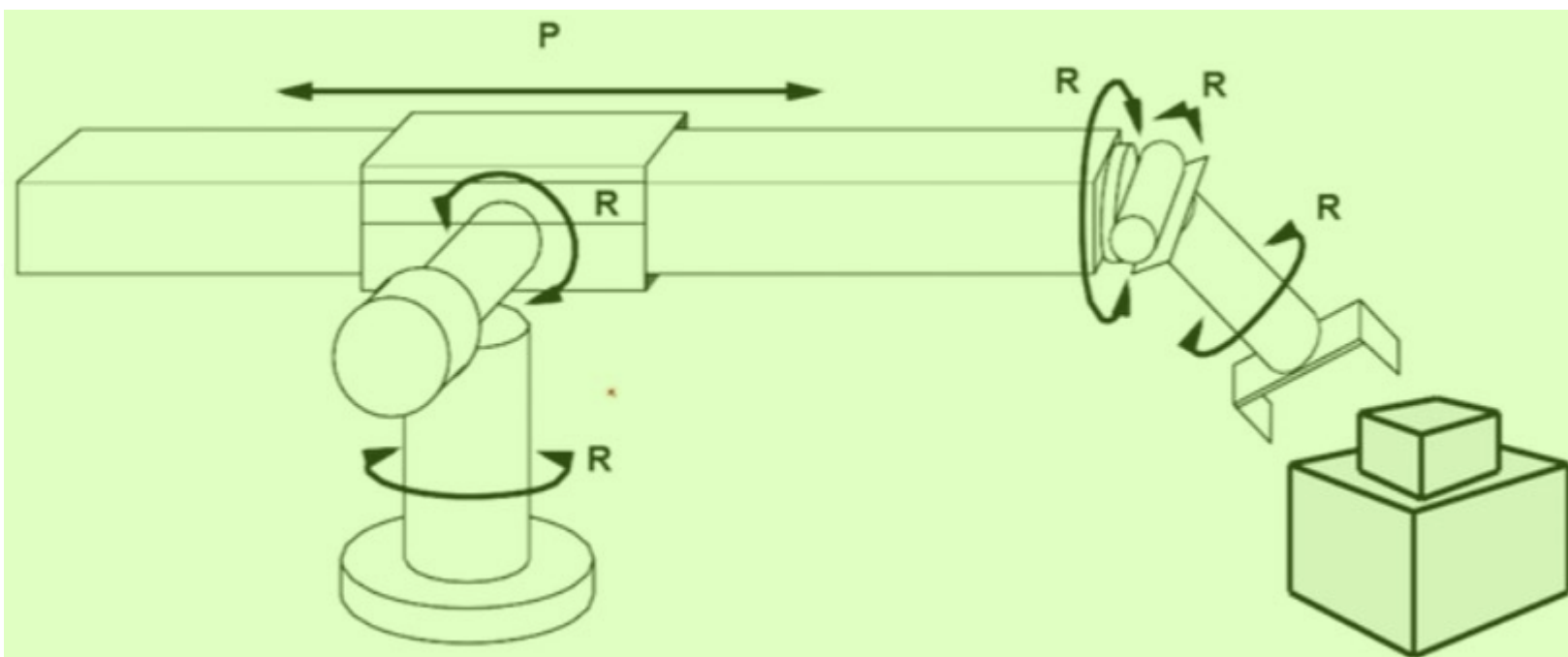


例5. 路径规划问题 (罗马尼亚)²

例5. 路径规划问题 (罗马尼亚)²

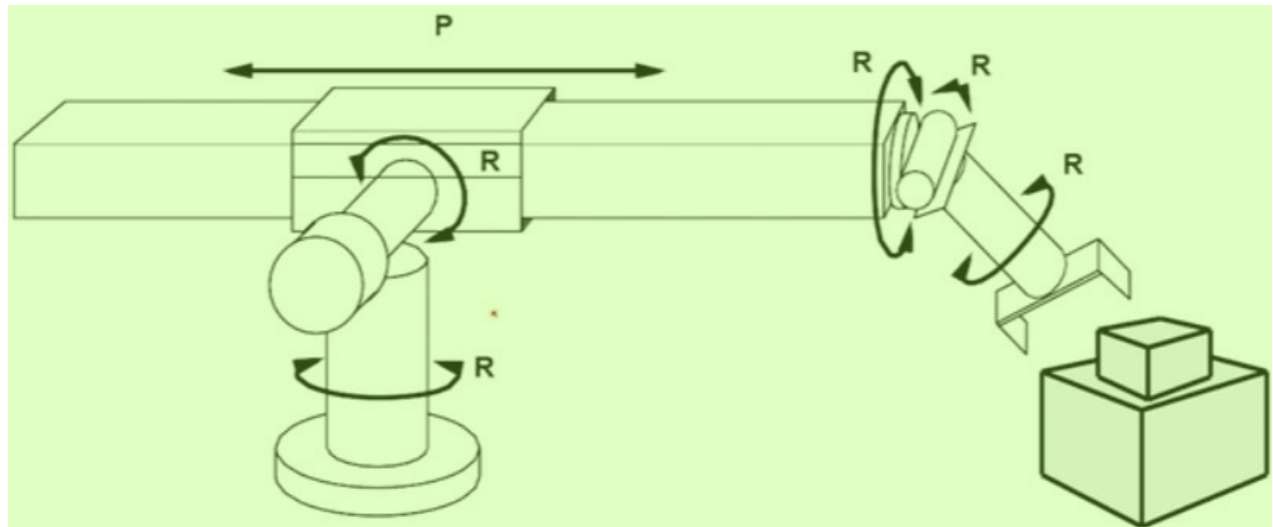
- 状态空间：城市节点³
 - 初始状态：某个节点⁴
 - 目标条件：某个或某些节点⁵
- 动作：去相邻城市⁶
- 状态转移函数：到达相邻城市⁷
- 路径代价函数：城市之间的距离⁸



例6. 机器人装配问题²

例6. 机器人装配问题²

- 初始状态：不限³
- 目标测试：检查是否将物体移动到正确位置⁴
- 动作：各个关节的运动⁵
- 转移模型：各个关节运动，机器人的姿态变化⁶
- 路径代价函数：所有关节运动总和⁷





搜索问题 (Problem)²

- 状态空间 环境状态的集合³
- 初始状态 智能体启动时的初始状态⁴
- 目标状态 智能体期望达到的终止状态⁵
- 动作集合 智能体可执行的动作⁶
- 转移模型 在状态 s 中执行动作 a 所产生的状态⁷
- 动作代价函数 在状态 s 中执行动作 a 转移到状态 s' 的数值代价⁸



问题的解 (Solution)²

- 路径³

动作序列⁷

- 解⁴

从初始状态到某个目标状态的路径⁸

- 路径代价⁵

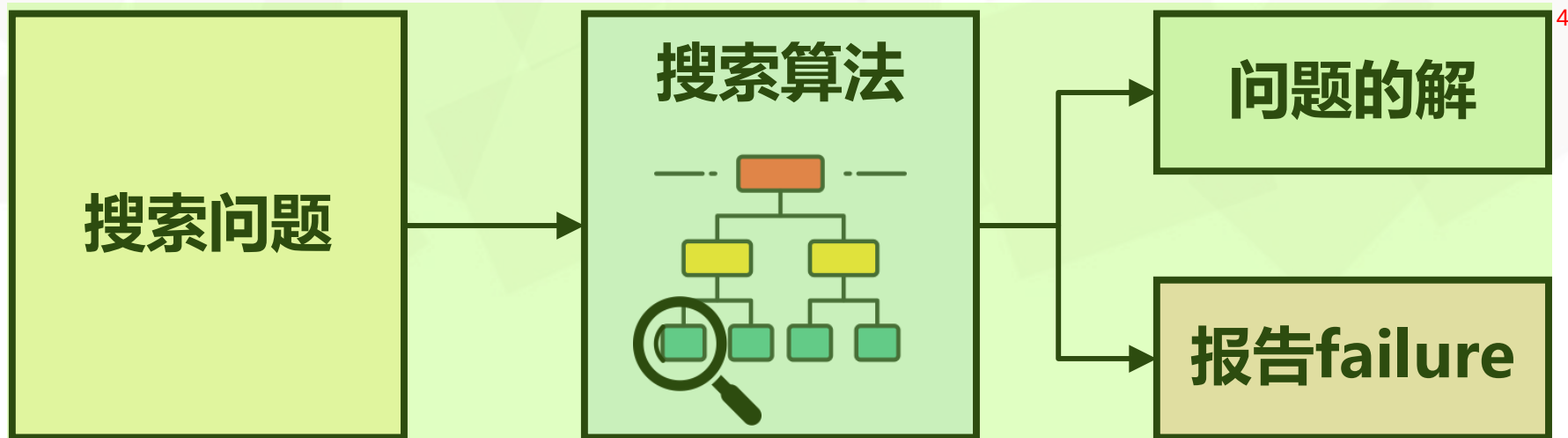
路径上各个动作代价的总和⁹

- 最优解⁶

所有解中路径代价最小的解¹⁰

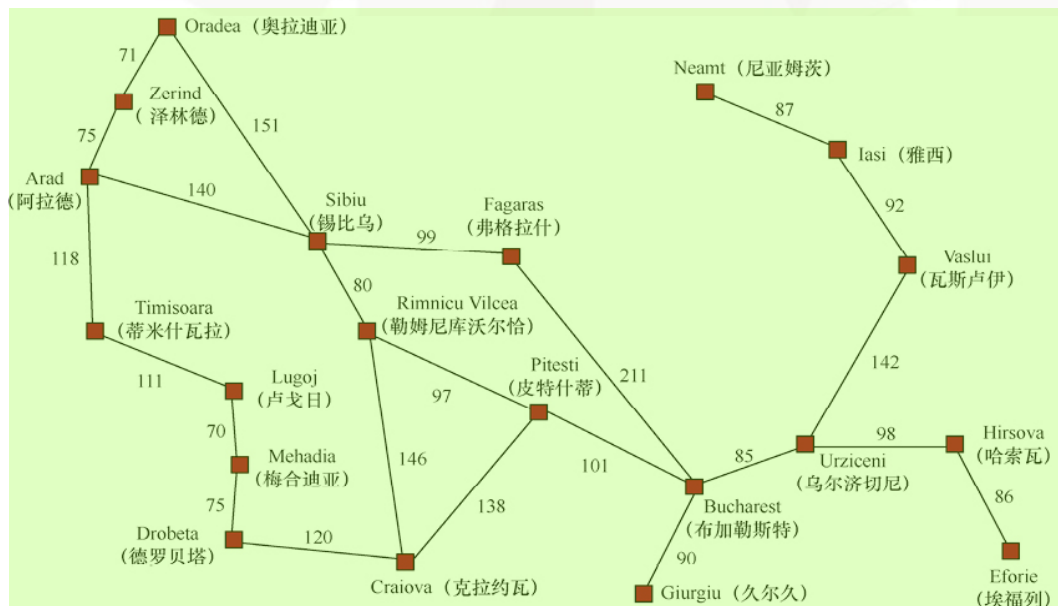
搜索算法²

- 将搜索问题作为输入，并返回问题的解或报告failure（当解不存在时）³

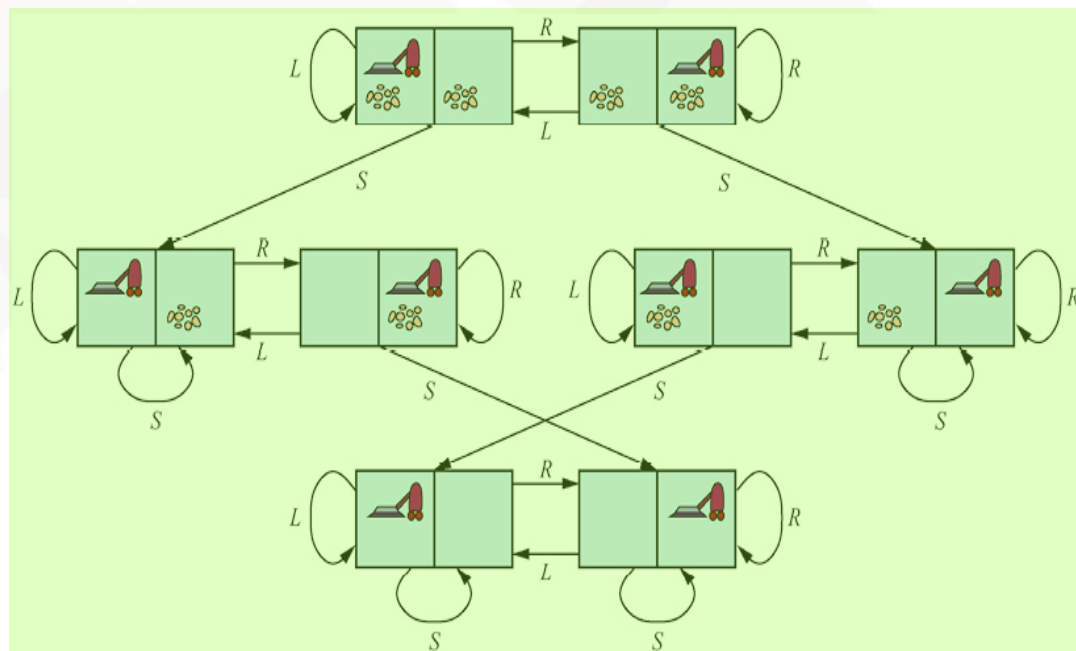


搜索算法²

- 状态空间图：用节点和边表示问题所有可能状态及状态间转换关系的图³
- 在状态空间图上叠加一棵搜索树⁴
- 从初始状态形成多条路径，并试图找到一条可以达到某个目标状态的路径⁵



6



8

搜索树²• 树³

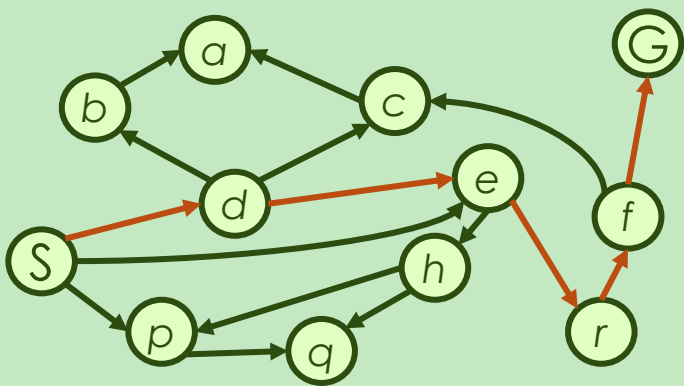
- 结点：状态⁴
- 边：动作/可选动作⁵
- 根结点：初始状态⁶
- 满足目标条件的叶结点：目标状态⁷

• 解⁸

- 从初始状态到达目标状态的动作序列⁹

状态空间图与搜索树的对应关系²

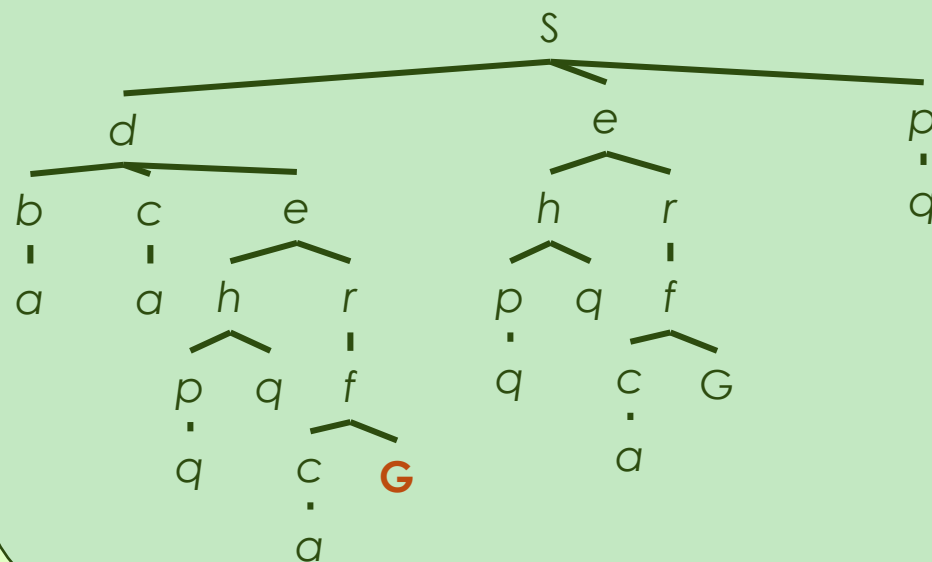
State Space Graph



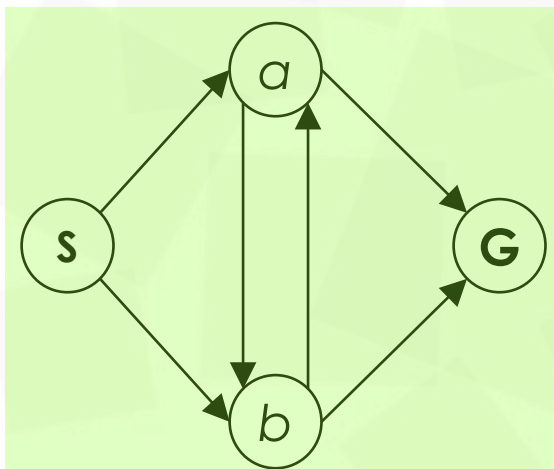
状态空间图描述了世界的状态集，以及允许从一个状态转移到另一个状态的动作

搜索树描述了这些状态之间通向目标的路径

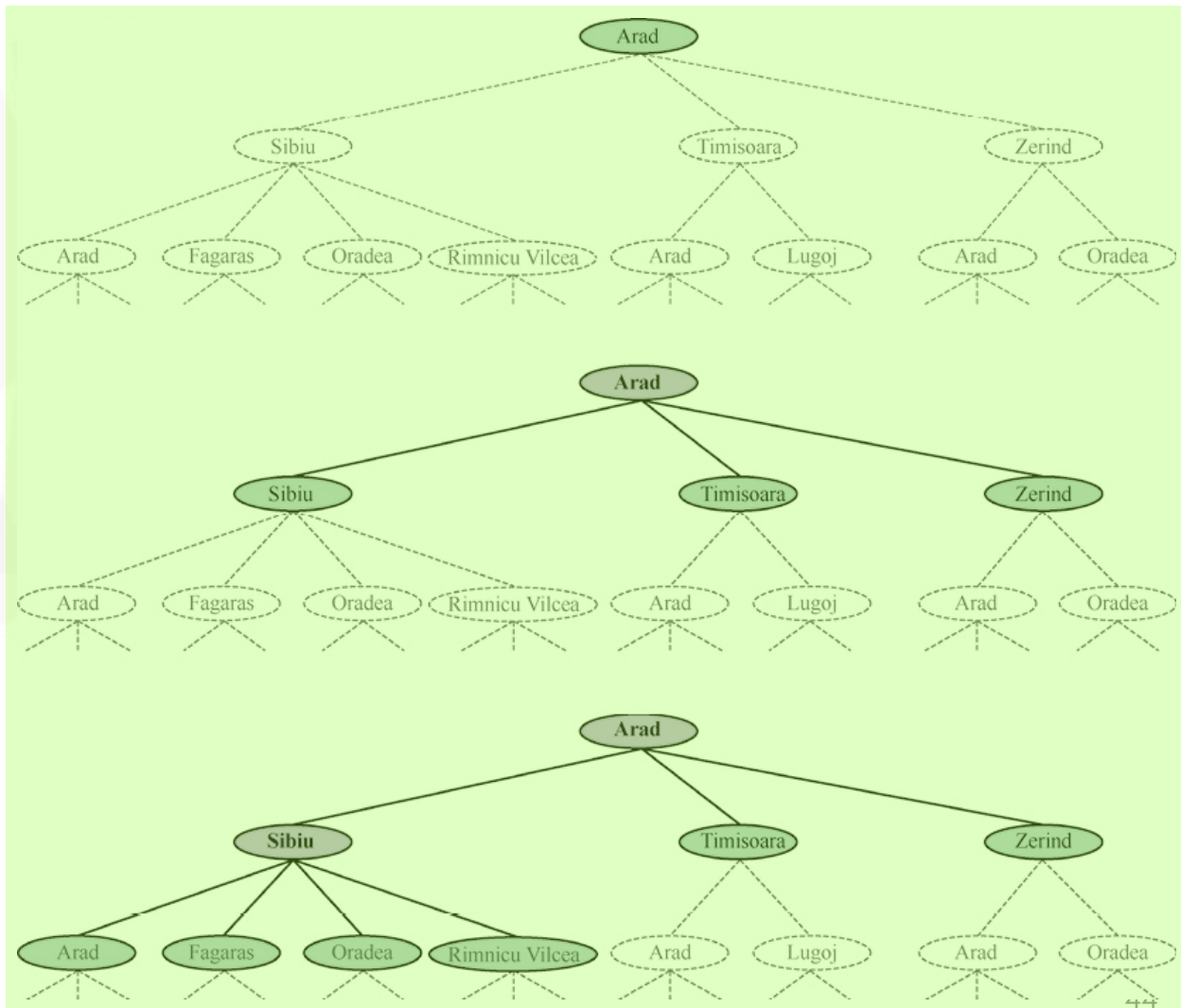
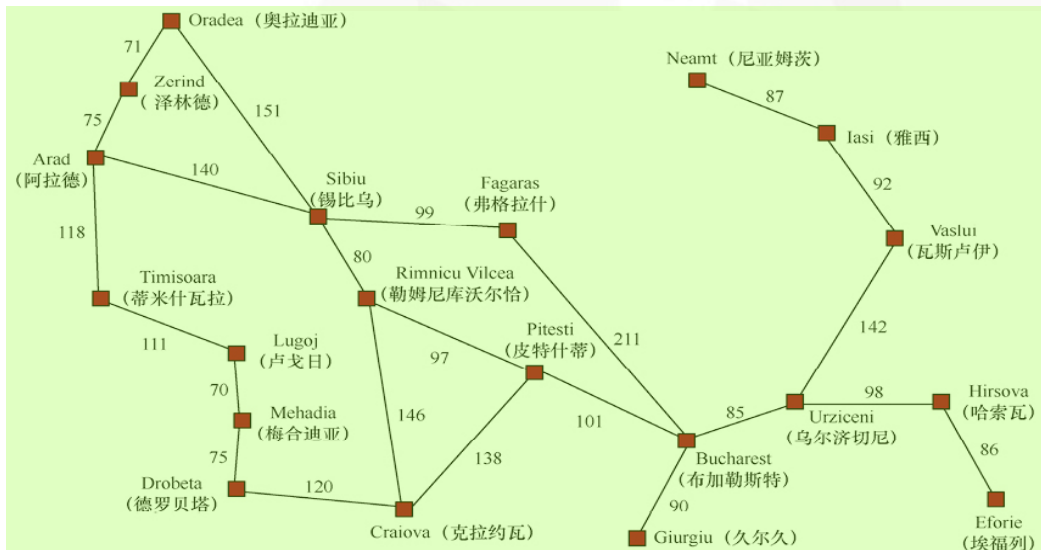
Search Tree



例. 对应的搜索树有多大?²



例. 路径规划问题的部分搜索树²



树搜索²

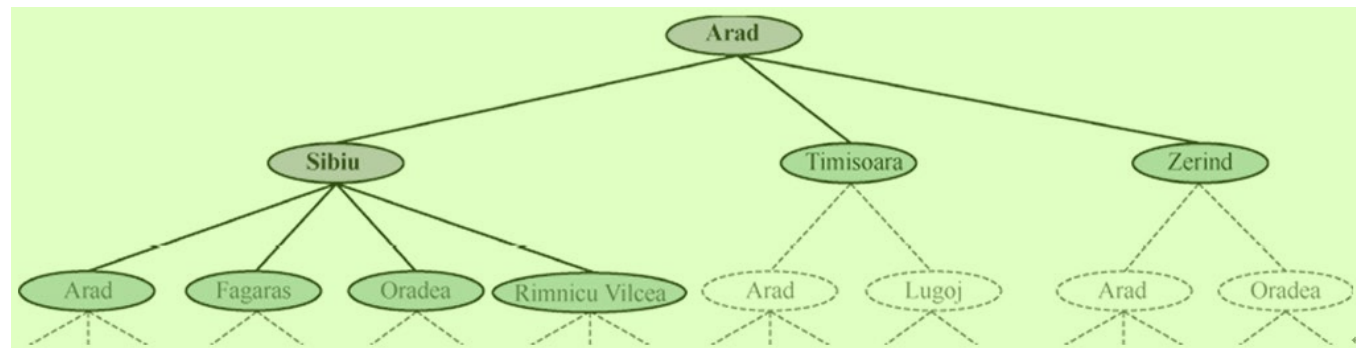
- 利用搜索树的树搜索³

- 初始状态⁴

- 动作/可选动作⁶

- 扩展：任一状态利用可选动作生成后继状态的过程⁷
 - 待扩展的（叶）结点：搜索树的边界（frontier）

- 搜索策略：从边界集选择哪个节点进行扩展⁸



5

树搜索算法²

function TREE-SEARCH(*problem*) returns a solution, or failure
 initialize the frontier using the initial state of *problem*
 loop do
 if the frontier is empty then return failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state then return the corresponding solution
 expand the chosen node, adding the resulting nodes to the frontier

例. 路径规划问题的搜索树²

• 路径中有重复状态/环路³

▸ Arad->Sibiu->Arad⁴

▸ Arad->Sibiu->Oradea->Zerind->Arad⁵

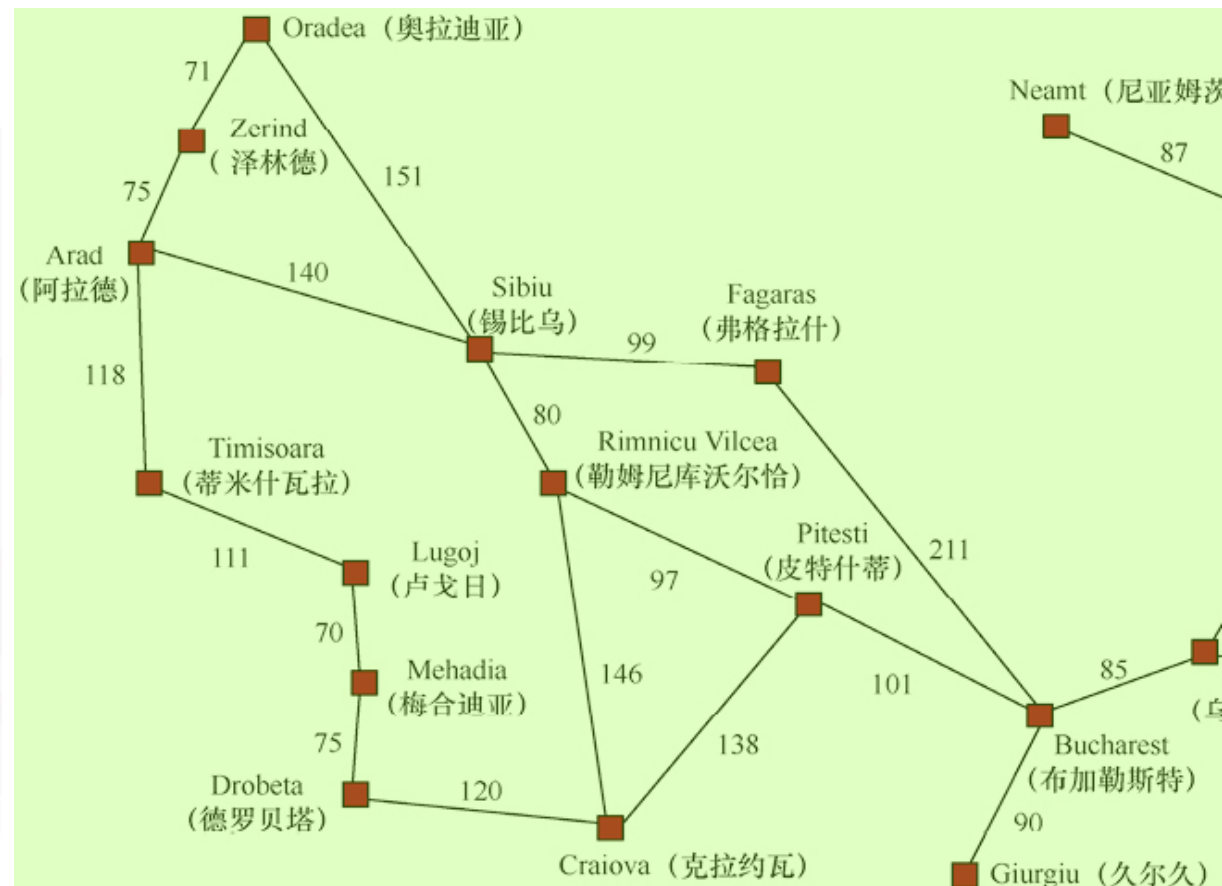
• 冗余路径⁶

▸ Sibiu->Arad (140)⁷

▸ Sibiu->Oradea->Zerind->Arad (297)⁸

• 搜索树是无穷深度的⁹

▸ 需要避免冗余状态或限制深度¹⁰



图搜索²

- 对于无向图搜索，冗余状态不可避免³
- 保存探索历史信息，避免探索冗余路径⁴
 - 树搜索算法上增加探索集，记录已扩展的结点⁵
 - 新生成的结点若与已生成的（探索集或边界集中的）某个结点相同，⁶则舍弃

图搜索²

- 利用搜索树的图搜索³

- 初始状态⁴

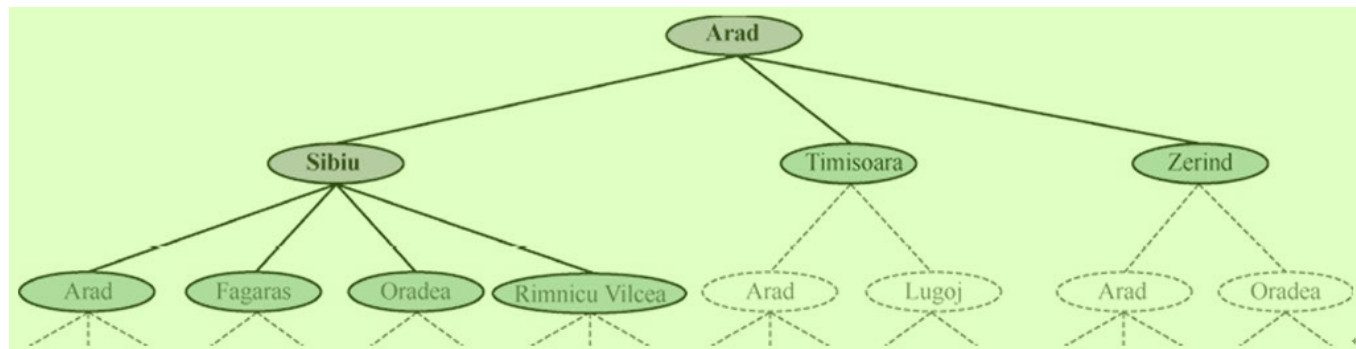
- 动作/可选动作⁵

- 扩展：任一状态利用可选动作生成后继状态的过程⁷

- 待扩展的（叶）结点：边界（frontier/open）⁸

- 已扩展的（中间）结点：探索（explored/reached/closed）⁹

- 搜索策略：从边界集选择哪个节点进行扩展，且该扩展的节点不在边界集或探索集中¹⁰



图搜索算法²

function GRAPH-SEARCH(*problem*) returns a solution, or failure
 initialize the frontier using the initial state of *problem*.
 initialize the explored set to be empty
 loop do
 if the frontier is empty then return failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state then return the corresponding solution
 add the node to the explored set
 expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set



function TREE-SEARCH(*problem*) returns a solution, or failure
initialize the frontier using the initial state of *problem*
loop do
 if the frontier is empty then return failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state then return the corresponding solution
 expand the chosen node, adding the resulting nodes to the frontier

function GRAPH-SEARCH(*problem*) returns a solution, or failure
initialize the frontier using the initial state of *problem*
initialize the explored set to be empty
loop do
 if the frontier is empty then return failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state then return the corresponding solution
 add the node to the explored set
 expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set

图 3.7 一般的树搜索和图搜索算法的非形式化描述。用粗斜体标出的图搜索算法的部分内容是指需要额外处理重复状态

搜索树中节点的数据结构²

- 搜索树中的结点 n ³

- $n.State$: 当前结点对应的状态⁴

- $n.Parent$: 当前结点的父结点⁵

- $n.Action$: 当前结点的父结点生成当前结点所采取的动作⁶

- $n.Path-Cost$: 从根结点到当前结点的路径代价值⁷

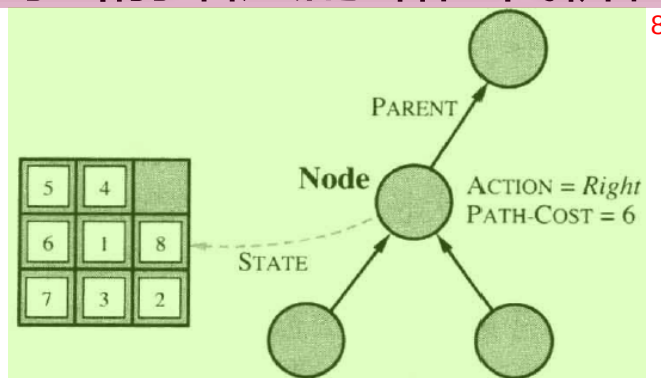


图 3.10 结点数据结构，由搜索树构造。每个结点都有一个父结点、一个状态和其他域。箭头由子结点指向父结点¹⁰



搜索相关的数据结构²

- 父结点通过执行动作扩展子节点³

```
function CHILD-NODE(problem, parent, action) returns a node4  
return a node with  
    STATE = problem.RESULT( parent.STATE, action),  
    PARENT = parent, ACTION = action,  
    PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

- 每个结点包含指向其父结点的指针 Parent⁵
- 利用该指针，可以从目标结点倒推出解路径⁶



节点 vs 状态²

- 节点：搜索树的数据结构³
- 状态：问题世界的描述⁴
- 同一状态可以通过不同的路径生成，则不同的节点可以表示相同的状态⁵



存放结点的常见数据结构²

队列（广义上的队列，可能是栈、队列、有限队列）³

- Is-Empty(frontier): 返回true当且仅当边界中没有节点⁴
- Pop(frontier): 返回边界中的第一个节点并将它从边界中删除⁵
- Top(frontier): 返回（但不删除）边界中的第一个节点⁶
- Add(node, frontier): 将节点插入队列中的适当位置⁷



存放结点的常见数据结构²

- 优先队列：首先弹出根据评价函数 f 计算得到的代价最小的节点，用于最佳优先搜索³
- FIFO队列：先进先出队列，首先弹出最先添加到队列中的节点，用于广度优先搜索⁴
- LIFO队列：后进先出队列，即栈，首先弹出最近添加的节点，用于深度优先搜索⁵



问题求解性能评估²

- 完备性：问题有解，算法一定能找到³
- 有效性：算法找到的解，是问题的解⁴
- 最优性：问题有解，算法一定能找到最优解⁵
- 时间复杂度：时间开销⁶
- 空间复杂度：空间开销⁷